CYBERSHIELD

Basic Cybersecurity Monitoring Dashboard

Mini Project Report

Technology: Python • Streamlit • pandas

Project type: Educational demonstration using synthetic data

Prepared for academic submission

Abstract

CyberShield is a lightweight, browser-based dashboard that demonstrates basic review of login activity. It presents fictional login records, summarizes successful and failed attempts, lets a user filter records, downloads the filtered history as a CSV file, and highlights IP addresses with repeated failed logins. The application is built using Python, Streamlit, and pandas and runs locally without a database.

1. Introduction

Organizations often review authentication logs to understand ordinary access patterns and investigate repeated failed sign-ins. This project illustrates a small part of that workflow in a simple interface suitable for beginners. The records included in this version are synthetic and must not be interpreted as evidence about real users or systems.

2. Problem Statement

Raw login events can be difficult for a beginner to inspect quickly. A simple dashboard can make the records easier to filter, summarize, export, and review for patterns that may deserve attention.

3. Objectives

• Display a set of 20 fictional login events in a table.

• Summarize total, successful, and failed login events.

• Show success/failure counts in a chart.

• Filter records by result, username, and IP address.

• Export filtered records as a CSV file.

• Demonstrate a basic rule for repeated failed attempts from the same IP address.

4. Tools and Technologies

Tool	Purpose
Python	Application logic
Streamlit	Browser-based user interface
pandas	Tabular data handling and summaries
CSV	Portable export format

5. System Design and Working

The application loads a static list of sample records into a pandas DataFrame. Sidebar selections are applied as filters. The filtered records feed the metric cards, chart, table, and CSV download. Separately, the application counts failed attempts per IP address in the full sample dataset and flags addresses with at least three failures.

Input: fictional login records. Processing: filtering, counting, and threshold comparison. Output: summary metrics, chart, history table, CSV export, and a suspicious-activity review table.

6. Main Features

• Dashboard metrics: total events, successful logins, failed logins, and failed rate.

• Login activity chart comparing successful and failed events.

• Search and filters for quick exploration.

• CSV download for records matching the selected filters.

• Repeated-failure review table using a demonstration threshold of 3 or more failed attempts.

7. Installation and Execution

Install Python 3.10 or newer. Open a terminal in the project folder and run:
`python -m venv .venv`
`.venv\\Scripts\\activate` (Windows; on macOS/Linux use `source .venv/bin/activate`)
`pip install -r requirements.txt`
`streamlit run app.py`
Open the local address printed by Streamlit, usually http://localhost:8501.

8. Testing

Test	Expected result
Launch app	Dashboard opens in the browser
Choose Failed only	Only failed login records remain in the filtered table
Search 203.0.113.45	Matching sample records are displayed
Download CSV	A CSV file containing filtered records is downloaded
Review suspicious IPs	IPs with at least 3 failed sample attempts are listed

9. Limitations

This prototype uses static synthetic records and does not ingest real logs, identify real attackers, block IP addresses, or provide production-grade security monitoring. The threshold is intentionally simple and can generate false positives. The example IP ranges are reserved for documentation/demo use.

10. Future Enhancements

Possible improvements include importing authorized log files, adding date/time filters, evaluating failed attempts within a defined time window, writing automated tests, adding user authentication and audit trails, and documenting privacy and retention policies.

Conclusion

CyberShield provides a basic hands-on demonstration of how a small dataset of login events can be summarized and reviewed. It meets the project's educational goals while keeping all activity local and fictional.

References

1. Streamlit documentation: https://docs.streamlit.io/
2. pandas documentation: https://pandas.pydata.org/docs/

3. Python documentation: https://docs.python.org/3/